CAD ENGINEERING TECHNICAL INTERVIEW

EDA / CAD Engineering Projects

Logic Synthesis / Physical Design / Timing ECO / CAD Flow Automation

Yu-Jui (Stanley) Liang
M.S. Candidate, Data Science Master Degree Program
National Taiwan University

liangyujui@gmail.com | linkedin.com/in/yu-jui-liang-735a57316

Education & Research Interests

Education

National Taiwan University
M.S. Candidate, Data Science Master Degree Program
Sep. 2025 – Expected Jun. 2027

National Chengchi University
B.S. in Computer Science
Double Major in Electronic Physics
Sep. 2021 – Jun. 2025

Research Interests

EDA / CAD & VLSI Design Automation

Logic Synthesis
Technology Mapping
Static Timing Analysis
Timing Closure
Physical Design
Incremental ECO
PPA

Algorithm & Software Development

C++ / Python / Tcl Flow Automation

Professional Software Engineering Experience

02 / WORK EXPERIENCE

Details index

Software development, simulation, and validation experience in R&D environments.

AIMobile Co., Ltd. (Inventec Group)

Apr. 2025 – Jul. 2025

Contract Software Engineer

Contributed to software development and system integration in an R&D environment.

Supported debugging and functional validation across software modules.

Neopower Technologies Co., Ltd.

Jun. 2023 – Mar. 2025

Contract Software Engineer

Developed a custom system simulator to reproduce operational and error scenarios.

Performed regression checks, log analysis, and customer-side A/B validation.

Selected Projects & Discussion Topics

Details index

03 / PROJECT OVERVIEW

01 Master's Research — Logic Synthesis (Front End)

Joint Single- and Multi-Output Cell Mapping via Differentiable Optimization

EDA Tool Development / Boolean Matching / Area–Delay Optimization

02 ICCAD 2025 Problem C — CAD Methodology (Back End)

STA-Driven Incremental Timing ECO Search

OpenROAD / OpenSTA / Python–Tcl Flow Automation / Heuristic PPA Search

PROJECT 01 / MASTER'S RESEARCH / LOGIC SYNTHESIS

Technology Mapping for Multi-Output Standard Cells

FA/HA cells over a shared input boundary

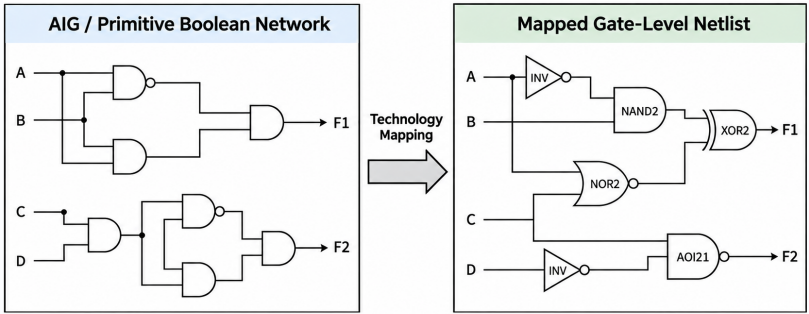
MY PROPOSED APPROACH

Joint Single- and Multi-Output Cell Mapping via Differentiable Optimization

MO candidate discovery in ABC, area-aware MILP pre-selection, and a mapping optimizer adapted from prior work.

ABC Mapper: SO Technology Mapper Baseline

The open-source ABC synthesis framework primarily targets single-output cell mapping, leaving joint FA/HA output mapping unsupported in the baseline flow.

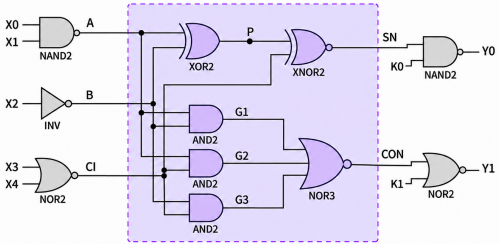


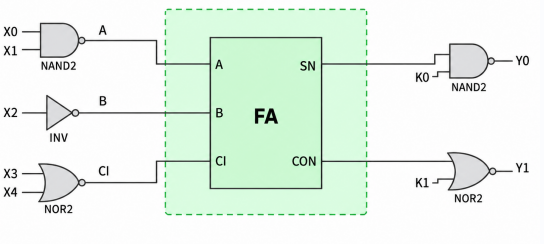
Same Boolean behavior, realized with ASAP7 7nm library cells.

Single-Output vs Multi-Output Cell Mapping

02 / MOTIVATION

A shared FA cell can implement two output functions over the same input boundary.

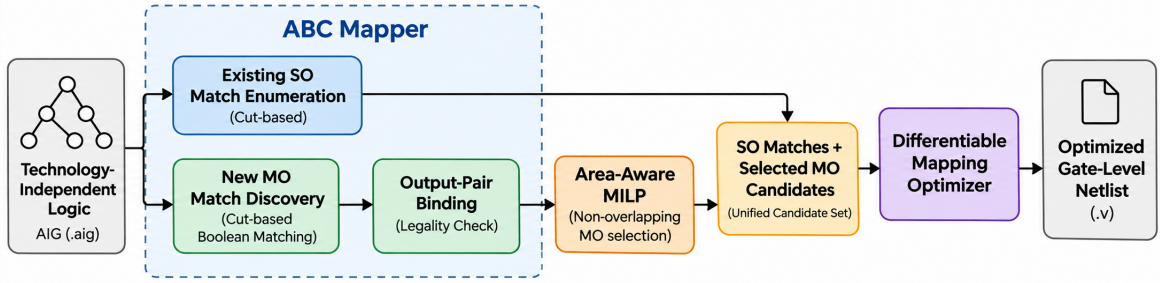




Independent MO discovery exposes joint output matches beyond SO-oriented cut retention.

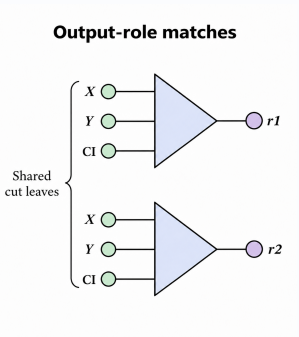
Joint SO / MO Mapping: Research Pipeline

We extend the ABC Mapper with multi-output candidate discovery and area-aware MILP selection, while preserving the original single-output mapping path.



MO Candidate Discovery in ABC

04 / CORE ALGORITHM



01 / MO Pattern Discovery

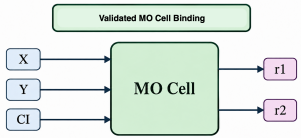
Search for MO-compatible logic patterns independently of SO-oriented pruning.

02 / Output Pair Matching

Match output-function pairs with shared inputs and compatible cell pin mapping.

03 / Candidate Validation

Verify functional and structural legality, then identify the SO logic that each MO cell could replace.

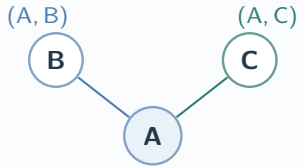


Area-Aware MO Selection (MILP)

Details index | SO witness | Verification

Maximize estimated area savings subject to non-overlapping MO candidates.

$$
\begin{aligned}
\max \quad & \sum_{i=1}^{N} (A_i^{SO} - A_i^{MO}) x_i \\
\text{s.t.} \quad & \sum_{i=1}^{N} a_{vi} x_i \le 1, \quad \forall v \in \mathcal{V} \\
& x_i \in \{0,1\}, \quad i = 1, \ldots, N
\end{aligned}
$$



$x_{AB} + x_{AC} \le 1$

Notation

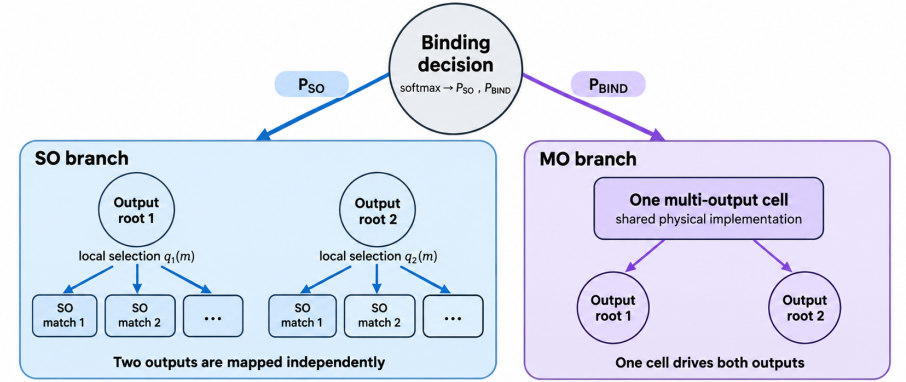
x_i: select MO i; A_i^{SO}: SO witness area; A_i^{MO}: MO cell area;

$a_{vi} = 1$ iff AIG base node v belongs to MO candidate i; otherwise 0.

Mapping Optimizer: SO / BIND Decisions

Details index Verification

MILP pre-selects non-overlapping MO candidates, while the mapping optimizer makes the final SO/BIND decisions.



Normalized ADP Results

Evaluation of four mapping configurations across 30 arithmetic logic circuit benchmarks using the ASAP7 7nm standard-cell library.

GMean Normalized ADP (Area–Delay Product)

ABC Mapper (SO Baseline)

1.000

Our Approach (SO Mode)



0.640

Our Approach (MO Mode without MILP, FIFO)



0.627

Our Approach (MO Mode with MILP)



0.602

FIFO rejects an MO candidate whose root is already used.

37.3% lower ADP than ABC Mapper

Our Approach MO mode (FIFO): 0.627 vs. 1.000

2% lower ADP than SO mode

Our Approach MO mode (FIFO): 0.627 vs. 0.640

6% lower ADP than SO mode

Our Approach MO mode (MILP): 0.602 vs. 0.640

Correctness & Scalability in Our Approach

Correctness

SO Mapping Regression

Preserve and test the original SO mapping flow after MO integration.

End-to-End CEC

Verify functional equivalence between the input logic and mapped netlist.

Scalability

Search-Space Limits

$K = 3$ maximum input signals per pattern.
$C = 128$ retained MO candidate patterns per node.

Runtime & Memory Control

Early pruning, deduplication, runtime and peak-memory monitoring.

PROJECT 02 / ICCAD 2025 CAD CONTEST / PROBLEM C

Incremental Placement Optimization Beyond Detailed Placement

Simultaneous Gate Sizing, Buffering, and Cell Relocation

MY PROPOSED APPROACH

STA-Driven Incremental Timing ECO Search

Use OpenROAD / OpenSTA with policy-controlled ECO moves and global PPA evaluation.

Multi-Objective Timing ECO Optimization

02 / PROBLEM DEFINITION

Details index | STA targets | QoR / budget

Improve setup TNS toward zero while controlling total power, HPWL, and cell displacement.

OPTIMIZATION OBJECTIVES

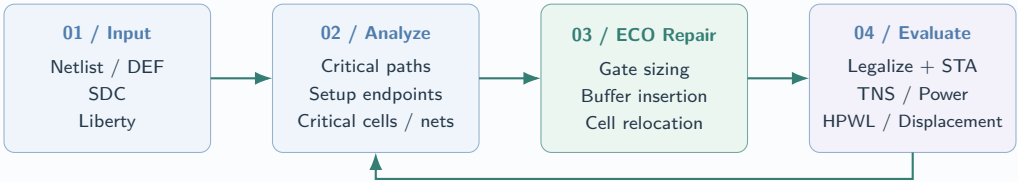
01 / Setup TNS
Reduce negative setup slack at violating endpoints.

02 / Total Power
Avoid excess power from ECO changes.

03 / HPWL
Control estimated wirelength after ECO.

04 / Displacement
Limit cell movement and keep placement legal.

BASIC ECO EVALUATION PIPELINE



Five Automatic ECO Strategy Candidates

02 / CANDIDATE SET

Details index | STA targets | Endpoint coverage | ECO sequences

The candidate is an automatic repair policy and editable region, not a manually chosen cell replacement.

S1 / Global Setup

Whole design; automatic upsizing and VT selection.

S2 / Path-Restricted

Top-K path cells + nets; localized setup repair.

S3 / Overlap-Aware

Top-I shared cells, nets fixed; sizing / VT repair.

S4 / Local Buffering

Top-K nets, original cells fixed; buffer insertion and load splitting.

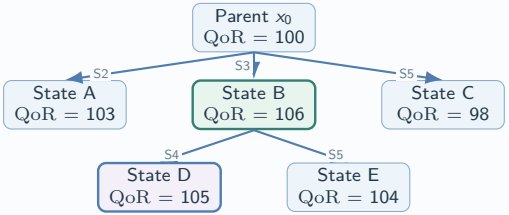
S5 / Power Recovery

Slack-rich cells / wider region; lower-power equivalent cells.

STA-Guided ECO Search: Best-First Branch List

No better child this round? Continue from the best unexplored state in the Branch List.

Search Tree Example



Bounded Branch List: D:105 > E:104 > A:103

Global Best: B (106) | **Next:** D (105)

A lower-scoring intermediate state may lead to a better ECO sequence later.

Heuristic Search Steps

01 / Pop Parent

Highest-QoR unexplored state from the **Branch List**.

02 / Generate + Evaluate

STA-guided ECO moves, legalization, and global QoR score.

03 / Update States

Save improved **Global Best**; queue legal child states.

04 / Next Branch

Expand best branch, **even with no new Global Best**.

QoR Results & Conclusion

02 / EARLIER ECO EVALUATION

Setup-focused timing repair and the resulting power / wirelength trade-off.

Setup Timing

16.7%
Median Setup TNS Improvement

14.6%
Average Setup WNS Improvement

Power & Wirelength

8%
Total Power Reduction

+2%
HPWL Increase

Conclusion

ECO operators can improve setup timing and power while accepting a small wirelength trade-off.

THANK YOU

Questions & Discussion

▸ Technical details

Technical Details

Back to project overview

QUESTIONS & DISCUSSION

Select a topic. Each detail page links back to the relevant summary slides.

Multi-Output Technology Mapping

D01 / FIFO candidate loss

D02 / Pair legality

D03 / SO witness

D04 / Verification

D10 / DC vs. ABC Synthesis Stages

STA-Driven Timing ECO

D05 / STA targets

D06 / Endpoint coverage

D07 / ECO sequences

D08 / QoR / budget

D09 / Parallel workers

D11 / Physical Design Flow

Baseline: File-Order Candidate Loss

File-order greedy filtering ignores estimated area benefits of conflicting MO candidates.

41,476
Original MO candidate BIND records

7,024
FIFO retained (offline simulation)

16.9%
Overall retention rate

Design	Original MO	FIFO Retained	Dropped	Retention
EPFL hyp	25,440	4,180	21,260	16.4%
EPFL square	3,658	544	3,114	14.9%
EPFL log2	2,798	400	2,398	14.3%
EPFL multiplier	2,190	349	1,841	15.9%
EPFL voter	632	106	526	16.8%
ISCAS c6288	258	37	221	14.3%
ISCAS c7552	102	11	91	10.8%
EPFL adder	6	1	5	16.7%
All 364 designs	**41,476**	**7,024**	**34,452**	**16.9%**

Source: MO-candidate census, 364 benchmark entries; simulated first-fit retention, not mapping-optimizer QoR.

MO Pair Matching & Structural Legality

05 / CANDIDATE VALIDATION

Individual legality is checked here; the MILP selects a conflict-free set.

Boolean Matching

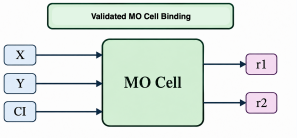
CON and SN roots match the cell's SUM and CARRY truth tables.

Pin Compatibility

Shared input support, with a compatible pin permutation and polarity.

Structural Legality

Distinct roots. Reject direct-dangling and TFI or cycle conflicts.



Individual MO Candidate Legality

Function, pins, structure, and the SO replacement witness are checked for one candidate. **Global Conflict-Free**

MO Selection

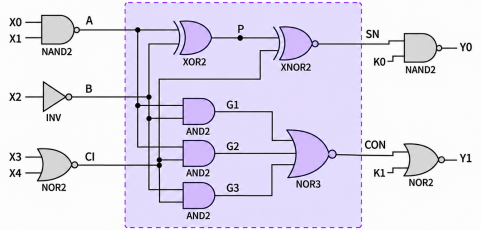
Overlapping candidates are resolved by the area-aware MILP, not by this check.

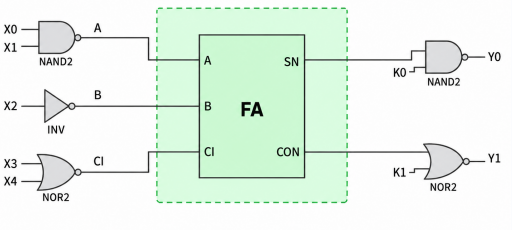
CON/SN name the two output roles. A legal record stores BIND, ROOTS, FANINS, and REPLACE_ROOTS.

SO Witness: Replacing Multiple SO Cells

06 / REPLACEMENT SEMANTICS

An MO candidate replaces a multi-level SO subcover with two output roots.



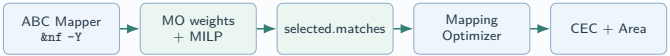


Trace the SO witness to the common MO input boundary. Record participating nodes as `REPLACE_ROOTS`; preserve any internal node still demanded externally.

The SO witness need not equal ABC's final selected SO cover.

Python + HiGHS: Verification Pipeline

Separate MILP pre-selection quality from the mapping optimizer's final mapped area.



The MILP filters MO candidates. SO matches from the ABC Mapper stay available to the optimizer.

Implementation

Python + HiGHS (`highspy`); emit selected.matches.

Correctness Checks

Gate A: unchanged ABC Mapper results
Gate B: individual MO legality
Gate C: optimizer materialization + CEC

Experimental Comparison

SO-only vs FIFO vs MILP-selected MO; same AIG, library, optimizer settings, and seed.

FIGURE SPACE

ADP and runtime chart

SO-only, FIFO, and MILP-selected MO, under the same settings.

Timing-Driven Target Extraction

Derive editable cells and nets from setup-violating paths and distinct endpoints.

```
report_checks -path_delay max
     -group_count 200
     -endpoint_count 3
     -slack_max 0
```

STA Report Scope

Setup paths may be PI→FF, FF→FF, FF→PO, or PI→PO, depending on constraints.

1 / Extract

Record endpoint, slack, path instances and nets.

2 / Deduplicate

Do not count the same violating endpoint repeatedly.

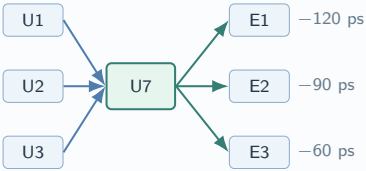
3 / Prioritize

Rank shared cells, critical nets, and candidate repair regions.

The STA report chooses where to search; OpenROAD performs the actual repair operation.

Slack-Weighted Endpoint Coverage

Prioritize shared cells by distinct endpoint coverage and violation severity.



One shared cell influences three distinct setup endpoints.

Distinct Endpoint Coverage

$Coverage(c) = |E(c)|$

Each endpoint is counted only once.

Slack-Weighted Priority

$$Priority(c) = \sum_{e \in E(c)} \max(0, -slack_e)$$

This priority guides search; the real ΔTNS still requires global STA after ECO.

Sequential ECO Strategy Combinations

Build a candidate by chaining compatible automatic repairs from a saved checkpoint.

S3 → S5

Repair shared timing-critical cells, then attempt power recovery if slack permits.

S4 → S3

Buffer selected critical nets, then re-target the remaining critical cells.

S3 → S4

Size critical cells first, then address residual net-delay bottlenecks.

Candidate Execution

Parent checkpoint

↓

Automatic repair pass 1

↓

Optional repair pass 2

↓

Legalize + re-evaluate global QoR

Operator order matters: buffering changes load; sizing changes driver strength.

Checkpoint Quality and Runtime Budget

Rank legal ECO checkpoints with consistent global QoR references, under one runtime budget.

ILLUSTRATIVE INTERNAL RANKING SCORE

$$\boxed{Q(x) = w_T\, g_T(x) + w_P\, g_P(x) - w_H\, p_H(x)}$$

$$g_T = \frac{|TNS_0| - |TNS_x|}{\max(|TNS_0|, \epsilon)}, \quad g_P = \frac{P_0 - P_x}{P_0}, \quad p_H = \frac{H_x - H_0}{H_0}$$

Quality-Based Search

Use the same baseline and weights when evaluating different branches; reject illegal or failed ECO states.

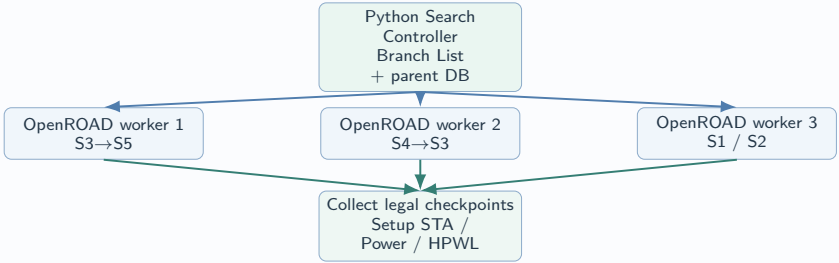
Runtime-Aware Execution

Batch processing consumes wall-clock time once; stop when the global execution budget is exhausted.

TNS: setup total negative slack; *P*: total power; *H*: HPWL. Weights define a search heuristic, not an official contest score.

Parallel Candidate Evaluation with OpenROAD

Independent workers run candidate ECO sequences from a selected Branch List checkpoint.



Worker Responsibilities

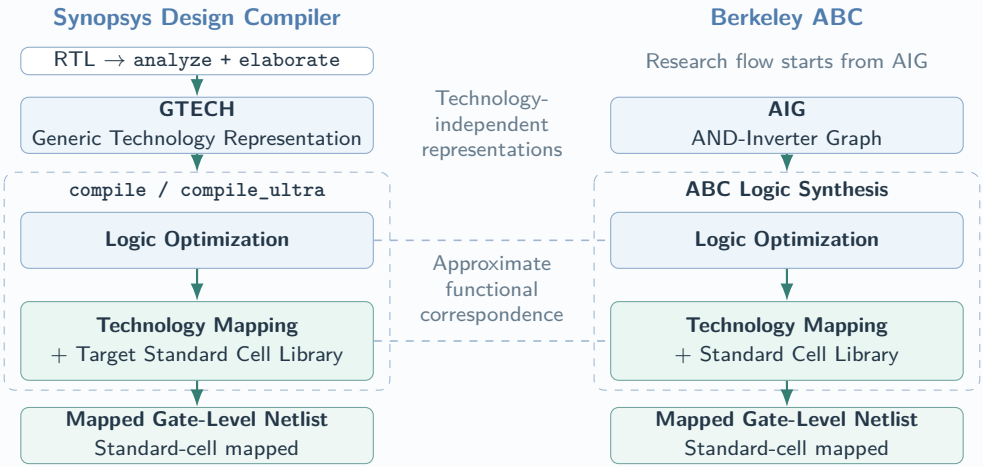
Apply ECO policy, legalize placement, update parasitics, collect full-design QoR, and save a restorable checkpoint.

Controller Responsibilities

Bound workers by available resources; update global best and Branch List using consistent QoR results.

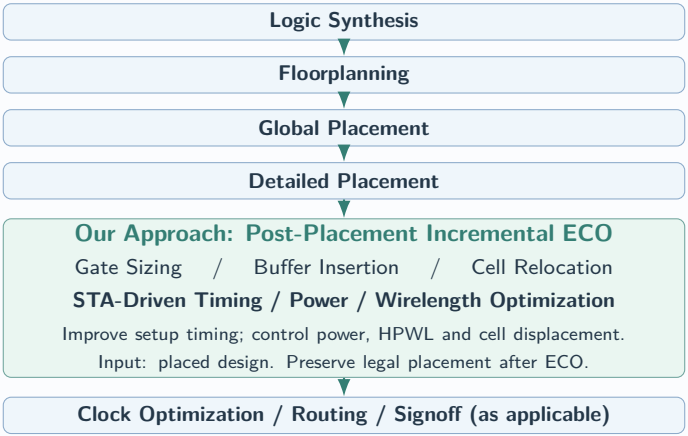
Design Compiler vs. ABC: Logic Synthesis Stages

D10 / LOGIC SYNTHESIS



Where Our ECO Fits in the Physical Design Flow

D11 / PHYSICAL DESIGN FLOW



Relative position only: clock-tree synthesis and timing optimization may interleave.